

- **Model Accuracy:**

- 1.Random Forest Regressor:**

- R2 Score: 0.7055499972053838

- MAE: 3.1930401724281117

- 2. Gradient Boosting :**

- R2 Score: 0.7704823220308483

- MAE: 2.774972657514048

- 3. SVM (Time-based Split) Results:**

- R2 Score: 0.8981127566374693

- MAE: 1.7328709934727813

- 4.SHAP Optimized Random Forest (SHAP used Feature Importance):**

- R2 Score: 0.6795688820808623

- MAE: 3.4559176635704616

- **Livability Feature Importance Analysis:**

Top influencing factors as per both **Random Forest Feature Importance** and **SHAP analysis** consistently highlight key features.

Selected Features:

- Education_Index
- Infrastructure_Index
- Healthcare_Index
- Urbanization_Score
- Environment_Infrastructure_Interaction

Random Forest have built-in **feature Importance Method** (model.feature_importances_) which only gives rank to each feature. Where as **SHAP (SHapley Additive Explanations)** is a method that not only ranks features but also explains how each feature contributes to the prediction by showing whether it increases or decreases the output and by how much.

- **Hyperparameters in Random Forest:**

```
n_estimators=700,  
max_depth=18,  
random_state=42,  
n_jobs=-1
```

n_estimators is number of trees, **max_depth** is the depth of tree, **random_state = 42** is used to fix the randomness in machine learning algorithms and its value can be any number from 0 to 100. **n_jobs = -1** means Use ALL available CPU cores and if it is **n_jobs = 1** means Use only 1 CPU core (slow).

- **Time-Based Split (≤ 2018 train, > 2018 test):**

1.Real-World Prediction

In real life, our system will do:

“Use past data $\rightarrow$ predict future livability”

So:

- Train = past (≤ 2018)
- Test = future (> 2018)

2.Suitable for Time-Series / Trend Data

Our dataset has:

- Year column
- Urban growth trends
- Economic & environmental changes

These are **time-dependent patterns**.

Random split breaks this pattern and Time split preserves it.

3. Prevents Data Leakage (VERY IMPORTANT)

If we use random split: Model may train on **future data (2020)** And test on **past data (2015)**. That's cheating, Model already “knows the future”. Time split avoids this.

- **Livability Index calculation:**

We categorized Indicators(factors) into Positive Indicators and Negative Indicators.

- **positive indicators**

Example: literacy, healthcare, infrastructure
Higher value = better livability

- **negative indicators**

Example: crime, pollution, unemployment

Lower value = better livability

But we used **df_neg = 1 - df_neg** which inverse the value and now we can say **“Higher value = better livability”**. It like if negative factor average is 0.45 so we took 1-0.45 and that is 0.55 so we want that 0.55 value should increase.

Then the **Formula is applied:**

```
df["Livability_Index"] = (df_pos.mean(axis=1) * 0.6 + df_neg.mean(axis=1) * 0.4) * 100
```

1. df_pos.mean(axis=1)

Takes **average of all positive indicators**.

2. df_neg.mean(axis=1)

Takes **average of negative indicators**.

3. Weighting

```
(df_pos * 0.6 + df_neg * 0.4)
```

Positive factors → **60% importance** and Negative factors → **40% importance**.

4. * 100

Converts score into Scale of **0 to 100** which is easy to interpret (like percentage).

Why this is used in project because we don't have a direct livability label so we created your own target variable and This is called **Index-based modelling**.

- **Classified Livability into categories:**

The predicted livability score is further classified into three qualitative categories:

Low: < 45

Medium: 45 – 55

High: ≥ 55

Why these specific thresholds?

This is based on **our dataset distribution** , Most of our livability cities scores are clustered around this range (45 to 55).

Inshort our Livability Index is centered around ~50.

- **WHY Random Forest is Selected instead of other models:**

Although SVM and Gradient Boosting (GB) is the best model on our dataset and they gives higher accuracy then Random Forest (RF) but we used Random Forest Because model selection is not only about accuracy.

Reasons to select Random Forest:

1.Interpretability (Biggest Advantages) :

- Random Forest:
Provides clear feature importance (**model.feature_importances_**).
Works directly with SHAP. Which feature matters and How much it matters.
- SVM:
It is a Black-box model (no direct interpretation)
- Gradient Boosting:
More complex and harder to interpret.

2. Stability & Robustness:

- Random Forest:
Builds trees independently (parallel) which gives stable predictions and Low overfitting risk.
Less sensitive to noise.
- Gradient Boosting:
Builds trees sequentially which can cause more can overfitting.
Sensitive to hyperparameters.
- SVM:
Highly sensitive to kernel choice, scaling, parameters.

3. Project Objective (VERY IMPORTANT):

Our project includes:

- Livability prediction
- Feature importance
- Policy simulation
- Decision support system

This requires: Understanding “**why**” , Not just “**what**”

- RF:
Supports explainability
- GB:
Less transparent
- SVM:
No explanation

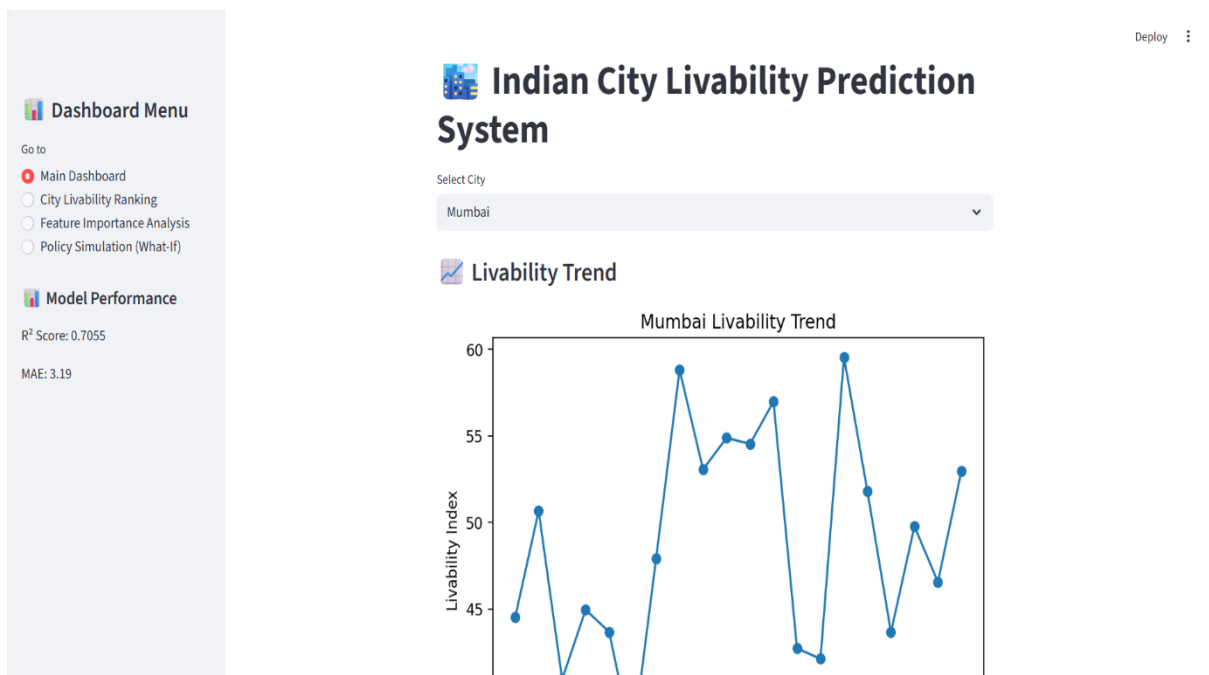
RF aligns best with decision-making systems.

4. Literature Support:

From our survey, Most papers Use RF or tree-based models so RF is academically justified.

Because it is the Balance between:

- Accuracy
- Interpretability



<<

Dashboard Menu

Go to

☐ Main Dashboard

☒ City Livability Ranking

☐ Feature Importance Analysis

☐ Policy Simulation (What-If)

Model Performance

R² Score: 0.7055

MAE: 3.19

Indian City Livability Prediction System

Select City

Mumbai



City Livability Ranking

	City	Predicted_Livability
0	Bhubaneswar	52.0384
1	Patna	51.2197
2	Chandigarh	51.2151
3	Kanpur	51.133
4	Vadodara	50.7693
5	Bhopal	50.7294
6	Lucknow	50.6759
7	Noida	50.6202
8	Chennai	50.5271
9	Nagpur	50.4738

Deploy

<<

Dashboard Menu

Go to

☐ Main Dashboard

☐ City Livability Ranking

☒ Feature Importance Analysis

☐ Policy Simulation (What-If)

Model Performance

R² Score: 0.7055

MAE: 3.19

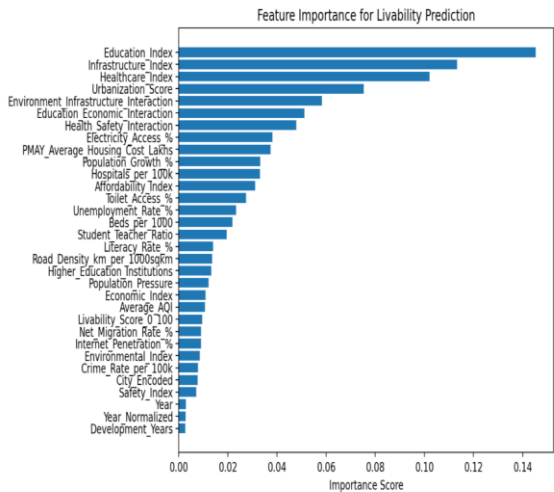
System

Select City

Mumbai



Feature Importance Analysis



Deploy

Dashboard Menu

Go to

- ☐ Main Dashboard
- ☐ City Livability Ranking
- ☐ Feature Importance Analysis
- ☒ Policy Simulation (What-If)

Model Performance

R² Score: 0.7055

MAE: 3.19

System

Select City

Mumbai

Policy Simulation (What-If Analysis)

Adjust key indicators to see how livability changes:



🏡 Simulated Livability Score for Mumbai : 49.82